



ASIGNACIÓN 7

Consultas Avanzadas

CÓDIGO SQL COMPLETO

Cada ejercicio se presenta como texto monoespaciado, sin recortes.

Estudiante: _____

Matrícula: _____

Oracle PL/SQL - Manejo de excepciones, procedimientos y funciones

```
-- EJERCICIO 9.1
SET SERVEROUTPUT ON
DECLARE
    v_apellido employees.last_name%TYPE;
    v_id        employees.employee_id%TYPE;
BEGIN
    SELECT last_name, employee_id
    INTO v_apellido, v_id
    FROM employees
    WHERE last_name = 'Cambrault'
    AND employee_id = (SELECT MIN(employee_id)
                       FROM employees
                       WHERE last_name = 'Cambrault');

    DBMS_OUTPUT.PUT_LINE('Apellido del empleado: ' || v_apellido);
    DBMS_OUTPUT.PUT_LINE('ID del empleado: ' || v_id);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No se encontro el empleado Cambrault.');
```

```
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Se encontro mas de un empleado Cambrault.');
```

```
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE(SQLERRM);
END;
/
```

```
-- EJERCICIO 9.2
SET SERVEROUTPUT ON
DECLARE
    e_campo_obligatorio EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_campo_obligatorio, -1400);
BEGIN
    INSERT INTO employees (employee_id, last_name)
    VALUES (NULL, 'Monasterio');
EXCEPTION
    WHEN e_campo_obligatorio THEN
        DBMS_OUTPUT.PUT_LINE('No se puede registrar la informacion...');
        DBMS_OUTPUT.PUT_LINE(SQLERRM);
END;
/
```

```
-- EJERCICIO 9.3
SET SERVEROUTPUT ON
BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE ct_errors PURGE';
EXCEPTION
    WHEN OTHERS THEN
        IF SQLCODE <> -942 THEN RAISE; END IF;
END;
/

CREATE TABLE ct_errors (
    e_user VARCHAR2(10),
    e_date DATE,
    e_code NUMBER(6),
    e_msg VARCHAR2(255)
);

DECLARE
    v_code NUMBER;
    v_msg VARCHAR2(255);
BEGIN
    INSERT INTO departments (department_id, department_name)
    VALUES (NULL, 'Software Architect');
EXCEPTION
    WHEN OTHERS THEN
        v_code := SQLCODE;
        v_msg := SQLERRM;
        INSERT INTO ct_errors VALUES (USER, SYSDATE, v_code, v_msg);
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('No se pudo registrar la informacion, mensaje de error grabado en la tabla CT_ERRORS');
END;
/

COLUMN e_user FORMAT A10
COLUMN e_date FORMAT A19
COLUMN e_msg FORMAT A65
SELECT e_user, TO_CHAR(e_date, 'DD/MM/YYYY HH24:MI:SS') e_date,
       e_code, e_msg
FROM ct_errors;
```

```
===== EJECUCIÓN EXITOSA =====

-- EJERCICIO 9.4 - ACTUALIZACION EXITOSA
SET SERVEROUTPUT ON
DEFINE p_employee_id = 100
DEFINE p_porcentaje = 5
DECLARE
    e_empleado_no_existe EXCEPTION;
BEGIN
    UPDATE employees
        SET salary = salary * (1 + &p_porcentaje / 100)
        WHERE employee_id = &p_employee_id;

    IF SQL%NOTFOUND THEN
        RAISE e_empleado_no_existe;
    END IF;

    DBMS_OUTPUT.PUT_LINE('La informacion fue actualizada con exito...');
    ROLLBACK;
EXCEPTION
    WHEN e_empleado_no_existe THEN
        DBMS_OUTPUT.PUT_LINE('El empleado no existe. Revise el Identificador del empleado!');
END;
/

===== EMPLEADO INEXISTENTE =====

-- EJERCICIO 9.4 - EMPLEADO INEXISTENTE
SET SERVEROUTPUT ON
DEFINE p_employee_id = 9999
DEFINE p_porcentaje = 5
DECLARE
    e_empleado_no_existe EXCEPTION;
BEGIN
    UPDATE employees
        SET salary = salary * (1 + &p_porcentaje / 100)
        WHERE employee_id = &p_employee_id;

    IF SQL%NOTFOUND THEN
        RAISE e_empleado_no_existe;
    END IF;

    DBMS_OUTPUT.PUT_LINE('La informacion fue actualizada con exito...');
    ROLLBACK;
EXCEPTION
    WHEN e_empleado_no_existe THEN
        DBMS_OUTPUT.PUT_LINE('El empleado no existe. Revise el Identificador del empleado!');
END;
/
```

```
-- EJERCICIO 9.5
SET SERVEROUTPUT ON
DECLARE
    e_departamento_no_existe EXCEPTION;
    e_cursor_invalido          EXCEPTION;
BEGIN
    DECLARE
        CURSOR c_localidades IS
            SELECT location_id, city
            FROM locations
            WHERE city IN ('Hiroshima', 'Oxford', 'South San Francisco')
            ORDER BY location_id;
        v_localidad locations.location_id%TYPE;
        v_ciudad     locations.city%TYPE;
    BEGIN
        OPEN c_localidades;
        LOOP
            FETCH c_localidades INTO v_localidad, v_ciudad;
            EXIT WHEN c_localidades%NOTFOUND;
            BEGIN
                UPDATE employees
                SET salary = salary * 1.03
                WHERE department_id IN (
                    SELECT department_id
                    FROM departments
                    WHERE location_id = v_localidad);

                IF SQL%NOTFOUND THEN
                    RAISE e_departamento_no_existe;
                END IF;

                DBMS_OUTPUT.PUT_LINE('Localidad ' || v_localidad ||
                    ' (' || v_ciudad || '): ' || SQL%ROWCOUNT ||
                    ' registros actualizados.');
```

```
            EXCEPTION
                WHEN e_departamento_no_existe THEN
                    DBMS_OUTPUT.PUT_LINE('Departamento no existe. Favor revise la localidad: ' || v_localidad);
            END;
        END LOOP;
        CLOSE c_localidades;
    EXCEPTION
        WHEN INVALID_CURSOR THEN
            RAISE e_cursor_invalido;
        WHEN OTHERS THEN
            IF c_localidades%ISOPEN THEN CLOSE c_localidades; END IF;
            RAISE;
    END;
    ROLLBACK;
EXCEPTION
    WHEN e_cursor_invalido THEN
        DBMS_OUTPUT.PUT_LINE('Acceso a cursor invalido!');
END;
/
```

```
-- EJERCICIO 9.6
SET SERVEROUTPUT ON
DEFINE p_apellido = ApellidoInexistente
BEGIN
    DELETE FROM employees
        WHERE last_name = '&p_apellido';

    IF SQL%NOTFOUND THEN
        RAISE_APPLICATION_ERROR(-20089, 'Apellido ingresado invalido');
    END IF;

    ROLLBACK;
END;
/
```

```
-- EJERCICIO 9.7
SET SERVEROUTPUT ON
DEFINE p_apellido = ApellidoInexistente
DECLARE
    e_apellido_invalido EXCEPTION;
BEGIN
    DELETE FROM employees
        WHERE last_name = '&p_apellido';

    IF SQL%NOTFOUND THEN
        RAISE e_apellido_invalido;
    END IF;

    ROLLBACK;
EXCEPTION
    WHEN e_apellido_invalido THEN
        RAISE_APPLICATION_ERROR(-20089, 'Apellido ingresado invalido');
END;
/
```



```
-- EJERCICIO 10.1
SET SERVEROUTPUT ON
CREATE OR REPLACE PROCEDURE actualizar_salario_3
  (p_employee_id IN employees.employee_id%TYPE) IS
  v_apellido employees.last_name%TYPE;
  v_anterior employees.salary%TYPE;
  v_nuevo     employees.salary%TYPE;
BEGIN
  SELECT last_name, salary
    INTO v_apellido, v_anterior
   FROM employees
  WHERE employee_id = p_employee_id;

  DBMS_OUTPUT.PUT_LINE('Antes: ' || v_apellido ||
    ' - ' || TO_CHAR(v_anterior, 'fm$99,999.00'));

  UPDATE employees SET salary = salary * 1.03
    WHERE employee_id = p_employee_id;

  SELECT salary INTO v_nuevo FROM employees
    WHERE employee_id = p_employee_id;

  DBMS_OUTPUT.PUT_LINE('Despues: ' || v_apellido ||
    ' - ' || TO_CHAR(v_nuevo, 'fm$99,999.00'));
END;
/

DEFINE p_employee_id = 100
BEGIN
  actualizar_salario_3(&p_employee_id);
  ROLLBACK;
END;
/
```

```
-- EJERCICIO 10.2
SET SERVEROUTPUT ON
CREATE OR REPLACE PROCEDURE detalle_empleado
(p_employee_id IN employees.employee_id%TYPE,
 p_apellido    OUT employees.last_name%TYPE,
 p_salario     OUT employees.salary%TYPE,
 p_fecha       OUT employees.hire_date%TYPE) IS
BEGIN
    SELECT last_name, salary, hire_date
    INTO p_apellido, p_salario, p_fecha
    FROM employees
    WHERE employee_id = p_employee_id;
END;
/

DEFINE p_employee_id = 100
DECLARE
    v_apellido employees.last_name%TYPE;
    v_salario  employees.salary%TYPE;
    v_fecha    employees.hire_date%TYPE;
BEGIN
    detalle_empleado(&p_employee_id, v_apellido, v_salario, v_fecha);
    DBMS_OUTPUT.PUT_LINE('Detalle del empleado: ' || v_apellido || ', ' ||
        TO_CHAR(v_salario, 'fm$99,999.00') || ', ' ||
        TO_CHAR(v_fecha, 'DD/MM/YYYY'));
END;
/
```

```
-- EJERCICIO 10.3
SET SERVEROUTPUT ON
CREATE OR REPLACE PROCEDURE raise_comm
  (p_employee_id IN employees.employee_id%TYPE,
   p_tasa        IN NUMBER) IS
BEGIN
  UPDATE employees
    SET salary = salary * (1 + p_tasa / 100)
    WHERE employee_id = p_employee_id;
END;
/

CREATE OR REPLACE PROCEDURE employee_proc IS
  CURSOR c_it_prog IS
    SELECT employee_id, last_name, salary
      FROM employees
     WHERE job_id = 'IT_PROG'
     ORDER BY employee_id;
  v_tasa NUMBER;
  v_nuevo employees.salary%TYPE;
BEGIN
  FOR r_emp IN c_it_prog LOOP
    IF r_emp.salary BETWEEN 2600 AND 6000 THEN
      v_tasa := 8;
    ELSIF r_emp.salary BETWEEN 6001 AND 7000 THEN
      v_tasa := 6;
    ELSIF r_emp.salary BETWEEN 7001 AND 8000 THEN
      v_tasa := 4;
    ELSE
      v_tasa := 2;
    END IF;

    raise_comm(r_emp.employee_id, v_tasa);
    SELECT salary INTO v_nuevo FROM employees
     WHERE employee_id = r_emp.employee_id;

    DBMS_OUTPUT.PUT_LINE('--- Datos del empleado ---');
    DBMS_OUTPUT.PUT_LINE('*** Apellido: ' || r_emp.last_name);
    DBMS_OUTPUT.PUT_LINE('*** Aumento [%]: ' || v_tasa || '%');
    DBMS_OUTPUT.PUT_LINE('*** Salario [V]: ' ||
      TO_CHAR(r_emp.salary, 'fm$99,999.00'));
    DBMS_OUTPUT.PUT_LINE('*** Salario [N]: ' ||
      TO_CHAR(v_nuevo, 'fm$99,999.00'));
  END LOOP;
END;
/

EXECUTE employee_proc;
ROLLBACK;
```

```
-- EJERCICIO 10.4
CREATE OR REPLACE FUNCTION tax
(p_salario IN employees.salary%TYPE)
RETURN NUMBER IS
BEGIN
    RETURN p_salario * 1.04;
END;
/

COLUMN salario_tax FORMAT $99,999.00
SELECT employee_id, tax(salary) AS salario_tax
FROM employees
WHERE tax(salary) > (
    SELECT MAX(tax(salary))
    FROM employees
    WHERE department_id = 80)
ORDER BY tax(salary);
```